

Lux Display Server: A Z Specification

Formal Model of Frame Lifecycle, Client Management, and Scene State

March 2026

Contents

1	Introduction	2
2	Basic Types	2
3	Free Types	2
4	Global Constants	2
5	Element	2
6	Scene	3
7	FrameReader	3
8	Display Server State	3
9	Initialization	4
10	Operations	4
10.1	Accept Connection	4
10.2	Disconnect Client	5
10.3	Receive Scene	5
10.4	Clear Scene	5
10.5	Remove Element (Patch)	5
10.6	Button Click (Interaction)	6
10.7	Flush Events	6
10.8	Shutdown	6
11	System Invariants	7

1 Introduction

The Lux display server is an ImGui render loop that listens on a Unix domain socket and renders scenes. Each frame, it accepts new connections, polls clients for messages, renders the current scene, and flushes interaction events.

This specification models the stateful core:

- Client connection tracking and FrameReader association
- Scene replacement, incremental patching, and clearing
- Message dispatch (scene, update, clear, ping)
- Interaction event generation and broadcast
- The FrameReader streaming buffer

We abstract away ImGui rendering (it is stateless from the protocol's perspective) and focus on the state that the protocol cares about.

2 Basic Types

$[FD, ELEMID, SCENEID, RAWBYTES, ACTION]$

FD models socket file descriptors. $ELEMID$ and $SCENEID$ are opaque identifiers for elements and scenes. $RAWBYTES$ represents a chunk of bytes received from a socket. $ACTION$ identifies the action string on a button click.

3 Free Types

$ZBOOL ::= ztrue \mid zfalse$

Element kinds supported in Phase 1:

$ElementKind ::= ekText \mid ekButton \mid ekSeparator \mid ekImage$

Client message types:

$ClientMsgType ::= cmScene \mid cmUpdate \mid cmClear \mid cmPing$

Display message types (sent back to clients):

$DisplayMsgType ::= dmReady \mid dmAck \mid dmPong \mid dmInteraction$

4 Global Constants

$maxClients : \mathbb{N}$ $maxElements : \mathbb{N}$ $maxEvents : \mathbb{N}$ $maxBufSize : \mathbb{N}$
$maxClients = 64$ $maxElements = 1000$ $maxEvents = 256$ $maxBufSize = 16777216$

5 Element

An element within a scene. We model only the identity and kind; the rendering attributes (content, label, path, etc.) are opaque to the protocol state machine.

$Element$ $elemId : ELEMID$ $elemKind : ElementKind$
--

6 Scene

A scene is an ordered list of elements with a unique scene ID. Element IDs within a scene must be unique.

<i>Scene</i>
<i>sceneId</i> : <i>SCENEID</i>
<i>elements</i> : seq <i>Element</i>
$\#elements \leq maxElements$
$\forall i, j : \text{dom } elements \bullet \quad i \neq j \Rightarrow \quad (elements\ i).elemId \neq (elements\ j).elemId$

7 FrameReader

The FrameReader accumulates raw bytes from a socket and tracks how many complete messages are available for draining.

We abstract the buffer as a natural number (bytes accumulated) and track how many complete messages have been parsed.

<i>FrameReaderState</i>
<i>bufSize</i> : $\mathbb{N}$
<i>pendingMsgs</i> : $\mathbb{N}$
$bufSize \leq maxBufSize$

<i>InitFrameReaderState</i>
<i>FrameReaderState'</i>
$bufSize' = 0$
$pendingMsgs' = 0$

Feeding bytes into the reader increases the buffer size.

<i>FeedBytes</i>
$\Delta FrameReaderState$
<i>bytesIn?</i> : $\mathbb{N}$
$bytesIn? > 0$
$bufSize + bytesIn? \leq maxBufSize$
$bufSize' = bufSize + bytesIn?$
$pendingMsgs' = pendingMsgs$

Draining extracts all complete messages, reducing the buffer. The number of messages drained is non-deterministic (depends on message boundaries within the byte stream).

<i>DrainMessages</i>
$\Delta FrameReaderState$
<i>drained!</i> : $\mathbb{N}$
<i>bytesConsumed?</i> : $\mathbb{N}$
$bytesConsumed? \leq bufSize$
$bufSize' = bufSize - bytesConsumed?$
$pendingMsgs' = 0$
$drained! = pendingMsgs + bytesConsumed?$

8 Display Server State

The server tracks connected clients, their FrameReaders, the current scene (if any), and a queue of pending interaction events.

DisplayServer

clients : $\mathbb{P} FD$
readers : $FD \rightarrow \text{FrameReaderState}$
hasScene : $ZBOOL$
currentScene : *Scene*
eventQueue : $\text{seq } ELEMID$
listening : $ZBOOL$

$\text{dom } readers = clients$
 $\#clients \leq \text{maxClients}$
 $\#eventQueue \leq \text{maxEvents}$
 $hasScene = \text{ztrue} \Rightarrow (\forall eid : \text{ran } eventQueue \bullet eid \in \{e : \text{ran}(currentScene.elements) \bullet e.elemId\})$

The key invariants are:

1. Every client has exactly one FrameReader ($\text{dom } readers = clients$).
2. Client count is bounded.
3. Queued events reference elements in the current scene.

9 Initialization

InitDisplayServer

DisplayServer'

clients' = $\emptyset$
readers' = $\emptyset$
hasScene' = zfalse
eventQueue' = $\langle \rangle$
listening' = ztrue

Init

InitDisplayServer

10 Operations

10.1 Accept Connection

A new client connects. We allocate a fresh FrameReader for it.

AcceptConnection

$\Delta DisplayServer$

newClient? : FD

listening = ztrue
newClient? $\notin clients$
 $\#clients < \text{maxClients}$
 $clients' = clients \cup \{newClient?\}$
 $(\exists frs : \text{FrameReaderState} \mid frs.bufSize = 0 \wedge frs.pendingMsgs = 0 \bullet readers' = readers \cup \{newClient? \mapsto frs\})$
hasScene' = *hasScene*
currentScene' = *currentScene*
eventQueue' = *eventQueue*
listening' = *listening*

10.2 Disconnect Client

A client disconnects (error or clean close).

<i>DisconnectClient</i> $\Delta DisplayServer$ <i>deadClient?</i> : <i>FD</i>
<i>deadClient?</i> $\in$ <i>clients</i> <i>clients'</i> = <i>clients</i> $\setminus$ { <i>deadClient?</i> } <i>readers'</i> = { <i>deadClient?</i> } $\Leftarrow$ <i>readers</i> <i>hasScene'</i> = <i>hasScene</i> <i>currentScene'</i> = <i>currentScene</i> <i>eventQueue'</i> = <i>eventQueue</i> <i>listening'</i> = <i>listening</i>

10.3 Receive Scene

A client sends a new scene, replacing any existing one. The event queue is cleared since old element references are stale.

<i>ReceiveScene</i> $\Delta DisplayServer$ <i>newScene?</i> : <i>Scene</i>
<i>hasScene'</i> = <i>ztrue</i> <i>currentScene'</i> = <i>newScene?</i> <i>eventQueue'</i> = $\langle \rangle$ <i>clients'</i> = <i>clients</i> <i>readers'</i> = <i>readers</i> <i>listening'</i> = <i>listening</i>

10.4 Clear Scene

A client sends a clear message, removing all content.

<i>ClearScene</i> $\Delta DisplayServer$
<i>hasScene'</i> = <i>zfalse</i> <i>eventQueue'</i> = $\langle \rangle$ <i>clients'</i> = <i>clients</i> <i>readers'</i> = <i>readers</i> <i>listening'</i> = <i>listening</i>

10.5 Remove Element (Patch)

An update message removes an element by ID from the current scene.

RemoveElement

$\Delta DisplayServer$
targetId? : *ELEMID*

hasScene = *ztrue*
targetId? $\in \{e : \text{ran}(\text{currentScene.elements}) \bullet e.\text{elemId}\}$
currentScene'.sceneId = *currentScene.sceneId*
 $\text{ran}(\text{currentScene'.elements}) = \{e : \text{ran}(\text{currentScene.elements}) \mid e.\text{elemId} \neq \text{targetId?}\}$
 $\# \text{currentScene'.elements} = \# \text{currentScene.elements} - 1$
hasScene' = *hasScene*
eventQueue' = *eventQueue*
clients' = *clients*
readers' = *readers*
listening' = *listening*

10.6 Button Click (Interaction)

The display detects a button click. An interaction event is queued for broadcast to all clients.

ButtonClick

$\Delta DisplayServer$
buttonId? : *ELEMID*

hasScene = *ztrue*
buttonId? $\in \{e : \text{ran}(\text{currentScene.elements}) \mid e.\text{elemKind} = \text{ekButton} \bullet e.\text{elemId}\}$
eventQueue' = *eventQueue* $\cap \langle \text{buttonId?} \rangle$
 $\# \text{eventQueue} < \text{maxEvents}$
hasScene' = *hasScene*
currentScene' = *currentScene*
clients' = *clients*
readers' = *readers*
listening' = *listening*

10.7 Flush Events

At the end of each frame, queued events are broadcast to all clients and the queue is emptied.

FlushEvents

$\Delta DisplayServer$

eventQueue' = $\langle \rangle$
hasScene' = *hasScene*
currentScene' = *currentScene*
clients' = *clients*
readers' = *readers*
listening' = *listening*

10.8 Shutdown

The display server shuts down: all clients are disconnected, the scene is cleared, events are flushed.

Shutdown

$\Delta DisplayServer$

clients' = $\emptyset$
readers' = $\emptyset$
hasScene' = *zfalse*
eventQueue' = $\langle \rangle$
listening' = *zfalse*

11 System Invariants

The key properties maintained across all operations:

1. **Reader–client bijection:** Every connected client has exactly one `FrameReader`, and every `FrameReader` belongs to a connected client ($\text{dom } \textit{readers} = \textit{clients}$).
2. **Event validity:** Queued interaction events reference elements that exist in the current scene.
3. **Bounded resources:** Client count, element count, event queue length, and buffer sizes are all bounded.
4. **Scene atomicity:** A new scene completely replaces the old one and clears stale events.
5. **Unique element IDs:** Within a scene, no two elements share an ID.